

第十一课预习报告

nogo.c

- 作用: 演示死循环程序的行为
- 功能: 程序进入死循环, 无法正常比较字符串
- 注意事项: 运行该程序会导致终端卡死
- 演示效果:

```
1  /* nogo.c -- will this work? */
2  #include <stdio.h>
3  #define ANSWER "Grant"
4  #define SIZE 40
5  char * s_gets(char * st, int n);
6
7  int main(void)
8  {
9      char try[SIZE];
10
11      puts("Who is buried in Grant's tomb?");
12      s_gets(try, SIZE);
13      while (try != ANSWER)
14      {
15          puts("No, that's wrong. Try again.");
16          s_gets(try, SIZE);
17      }
18      puts("That's right!");
19
20      return 0;
21  }
22
23  char * s_gets(char * st, int n)
24  {
25      char * ret_val;
26      int i = 0;
27
28      ret_val = fgets(st, n, stdin);
29      if (ret_val)
30      {
```

```

31         while (st[i] != '\n' && st[i] != '\0')
32             i++;
33         if (st[i] == '\n')
34             st[i] = '\0';
35         else // must have words[i] == '\0'
36             while (getchar() != '\n')
37                 continue;
38     }
39     return ret_val;
40 }

```

nono.c

- 作用: 演示字符数组与字符串的区别
- 功能: 定义三个字符数组, 其中 `dont` 数组不是以 `\0` 结尾的字符串, 直接用 `puts` 输出会导致继续读取后续内存内容
- 语义错误: `dont` 数组未以空字符结尾, `puts` 会越界读取, 输出包含后续内存数据
- 演示效果:

```

1  /* nono.c -- no! */
2  #include <stdio.h>
3  int main(void)
4  {
5      char side_a[] = "Side A";
6      char dont[] = {'W', '0', 'W', '!' };
7      char side_b[] = "Side B";
8
9      puts(dont);    /* dont is not a string */
10
11     return 0;
12 }

```

```

1  gcc nono.c -o nono && ./nono
2  WOW!Side A

```

p_and_s.c

- 作用: 演示指针与字符串的关系
- 功能: 定义两个指针变量指向同一个字符串常量, 输出它们的值、地址以及指向的内存地址, 展示

指针赋值与共享数据的效果

- 演示效果:

```
1  /* p_and_s.c -- pointers and strings */
2  #include <stdio.h>
3  int main(void)
4  {
5      const char * mesg = "Don't be a fool!";
6      const char * copy;
7
8      copy = mesg;
9      printf("%s\n", copy);
10     printf("mesg = %s; &mesg = %p; value = %p\n",
11           mesg, &mesg, mesg);
12     printf("copy = %s; &copy = %p; value = %p\n",
13           copy, &copy, copy);
14
15     return 0;
16 }
```

```
1  gcc p_and_s.c -o p_and_s && ./p_and_s
2  Don't be a fool!
3  mesg = Don't be a fool!; &mesg = 0000006f767ffb88; value = 00007ff754ada000
4  copy = Don't be a fool!; &copy = 0000006f767ffb80; value = 00007ff754ada000
```

put_out.c

- 作用: 演示 `puts()` 函数在处理不同类型字符串时的表现
- 功能: 展示字符串常量、数组以及指针三种形式在 `puts()` 输出中的差异, 通过偏移指针或数组地址实现部分输出
- 演示效果:

```
1  /* put_out.c -- using puts() */
```

```
1 gcc put_out.c -o put_out && ./put_out
2 I'm an argument to puts().
3 I am a #defined string.
4 An array was initialized to me.
5 A pointer was initialized to me.
6 ray was initialized to me.
7 inter was initialized to me.
```

put_put.c

- 作用: 演示自定义输出函数 `put1()` 和 `put2()` 的实现
- 功能: `put1()` 用循环逐字符输出字符串; `put2()` 在输出每个字符同时计数, 返回输出字符数
- 演示效果:

```
1 //put_put.c -- user-defined output functions
```

```
1 gcc put_put.c -o put_put && ./put_put
2 If I'd as much money as I could spend,
3 I never would cry old chairs to mend.
4 I count 37 characters.
```

quit_chk.c

- 作用: 演示用字符串检测循环退出条件
- 功能: 从用户输入最多10行字符串, 当检测到某行输入为“quit”时停止输入并输出输入统计
- 演示效果:

```
1 /* quit_chk.c -- reading input until "quit" */
```

```
1 gcc quit_chk.c -o quit_chk && echo "Hello\nWorld\nquit" | ./quit_chk
2 Enter up to 10 lines (type quit to quit):
3 1 strings entered
```

repeat.c

- 作用: 演示命令行参数的读取与统计
- 功能: 输出命令行参数的数量及内容, 展示 `argc` 和 `argv` 的基本用法
- 演示效果:

```

1  /* repeat.c -- echoing command-line arguments */
2  #include <stdio.h>
3  int main(int argc, char * argv[])
4  {
5      int count;
6
7      printf("The command line has %d arguments:\n", argc - 1);
8      for (count = 1; count < argc; count++)
9          printf("%d: %s\n", count, argv[count]);
10
11     return 0;
12 }

```

```

1  gcc repeat.c -o repeat && ./repeat
2  The command line has 0 arguments:

```

```

#include <stdio.h>
#include <string.h>
#define SIZE 80
#define LIM 10
char * s_gets(char * st, int n);

int main(void)
{
    char input[LIM][SIZE];
    int ct = 0;

```

```

1  printf("Enter up to %d lines (type quit to quit):\n", LIM);
2  while (ct < LIM && s_gets(input[ct], SIZE) != NULL && strcmp(input[ct], "quit")
   != 0)
3      ct++;
4  printf("%d strings entered\n", ct);
5
6  return 0;

```

```

}

char * s_gets(char * st, int n)
{
    char * ret_val;
    int i = 0;
    ret_val = fgets(st, n, stdin);
    if (ret_val)
    {
        while (st[i] != '\n' && st[i] != '\0')
            i++;
        if (st[i] == '\n')
            st[i] = '\0';
        else
            while (getchar() != '\n')
                continue;
    }
    return ret_val;
}

```

```

1  `` ` bash
2  gcc quit_chk.c -o quit_chk && echo "Hello\nWorld\nquit" | ./quit_chk
3  Enter up to 10 lines (type quit to quit):
4  1 strings entered

```

```

#include <stdio.h>
void put1(const char *);
int put2(const char *);

int main(void)
{
    put1("If I'd as much money");
    put1(" as I could spend,\n");
    printf("I count %d characters.\n",
        put2("I never would cry old chairs to mend."));

```

```

1  return 0;

```

```

}

```

```
void put1(const char * string)
{
    while (string) / same as *string != '\0' /
        putchar(string++);
}
```

```
int put2(const char * string)
{
    int count = 0;
    while (string)
    {
        putchar(string++);
        count++;
    }
    putchar('\n');
```

```
1 | return(count);
```

```
}
```

```
1 | `` `bash
2 | gcc put_put.c -o put_put && ./put_put
3 | If I'd as much money as I could spend,
4 | I never would cry old chairs to mend.
5 | I count 37 characters.
```

```
#include <stdio.h>
#define DEF "I am a #defined string."
int main(void)
{
    char str1[80] = "An array was initialized to me.";
    const char * str2 = "A pointer was initialized to me.";
```

```

1 puts("I'm an argument to puts().");
2 puts(DEF);
3 puts(str1);
4 puts(str2);
5 puts(&str1[5]);
6 puts(str2+4);
7
8 return 0;

```

```

}

```

```

1 `` `bash
2 gcc put_out.c -o put_out && ./put_out
3 I'm an argument to puts().
4 I am a #defined string.
5 An array was initialized to me.
6 A pointer was initialized to me.
7 ray was initialized to me.
8 inter was initialized to me.

```

- 作用: 演示指针与字符串的关系
- 功能: 定义两个指针变量指向同一个字符串常量，输出它们的值、地址以及指向的内存地址，展示指针赋值与共享数据的效果
- 演示效果:

```

1  /* p_and_s.c -- pointers and strings */
2  #include <stdio.h>
3  int main(void)
4  {
5      const char * mesg = "Don't be a fool!";
6      const char * copy;
7
8      copy = mesg;
9      printf("%s\n", copy);
10     printf("mesg = %s; &mesg = %p; value = %p\n",
11           mesg, &mesg, mesg);
12     printf("copy = %s; &copy = %p; value = %p\n",
13           copy, &copy, copy);
14
15     return 0;

```

```

1 gcc p_and_s.c -o p_and_s && ./p_and_s
2 Don't be a fool!
3 msg = Don't be a fool!; &msg = 0000006f767ffb88; value = 00007ff754ada000
4 copy = Don't be a fool!; &copy = 0000006f767ffb80; value = 00007ff754ada000

```

- 作用: 演示字符数组与字符串的区别
- 功能: 定义三个字符数组, 其中 `dont` 数组不是以 `\0` 结尾的字符串, 直接用 `puts` 输出会导致继续读取后续内存内容
- 语义错误: `dont` 数组未以空字符结尾, `puts` 会越界读取, 输出包含后续内存数据
- 演示效果:

```

1  /* nono.c -- no! */
2  #include <stdio.h>
3  int main(void)
4  {
5      char side_a[] = "Side A";
6      char dont[] = {'W', 'O', 'W', '!' };
7      char side_b[] = "Side B";
8
9      puts(dont);  /* dont is not a string */
10
11     return 0;
12 }

```

```

1 gcc nono.c -o nono && ./nono
2 WOW!Side A

```

input.txt

- 作用: 提供程序运行时的输入数据文件
- 功能: 包含多行字符串, 用于测试 `fgets()`、`gets()` 等读取函数的行为
- 内容:

```
1 Hello
2 World
3
```

getspu.ts.c

- 作用: 演示 `gets()` 和 `puts()` 的基本用法
- 功能: 从标准输入读取一行字符串并输出两次, 展示 `gets()` 的典型用法 (注意: `gets()` 不安全, 可能导致缓冲区溢出)
- 演示效果:

```
1  /* getspu.ts.c  -- using gets() and puts() */
2  #include <stdio.h>
3  #define STLEN 81
4  int main(void)
5  {
6      char words[STLEN];
7
8      puts("Enter a string, please.");
9      gets(words); // typical use
10     printf("Your string twice:\n");
11     printf("%s\n", words);
12     puts(words);
13     puts("Done.");
14
15     return 0;
16 }
```

```
1  gcc getspu.ts.c -o getspu.ts && echo "HelloWorld" | ./getspu.ts
2  Enter a string, please.
3  Your string twice:
4  HelloWorld
5  HelloWorld
6  Done.
```

format.c

- 作用: 演示 `sprintf()` 格式化字符串的用法

- 功能: 从用户输入获取名字、姓氏和奖金金额, 使用 `sprintf()` 将它们格式化为指定宽度和精度的字符串, 并输出结果
- 演示效果:

```
1  /* format.c -- format a string */
2  #include <stdio.h>
3  #define MAX 20
4  char * s_gets(char * st, int n);
5
6  int main(void)
7  {
8      char first[MAX];
9      char last[MAX];
10     char formal[2 * MAX + 10];
11     double prize;
12
13     puts("Enter your first name:");
14     s_gets(first, MAX);
15     puts("Enter your last name:");
16     s_gets(last, MAX);
17     puts("Enter your prize money:");
18     scanf("%lf", &prize);
19     sprintf(formal, "%s, %-19s: $%.2f\n", last, first, prize);
20     puts(formal);
21
22     return 0;
23 }
24
25 char * s_gets(char * st, int n)
26 {
27     char * ret_val;
28     int i = 0;
29
30     ret_val = fgets(st, n, stdin);
31     if (ret_val)
32     {
33         while (st[i] != '\n' && st[i] != '\0')
34             i++;
35         if (st[i] == '\n')
36             st[i] = '\0';
37         else // must have words[i] == '\0'
38             while (getchar() != '\n')
39                 continue;
```

```
40     }
41     return ret_val;
42 }
```

```
1 gcc format.c -o format && echo "John`nDoe`n1234.56" | ./format
2 Enter your first name:
3 Enter your last name:
4 Enter your prize money:
5 Doe, John           : $1234.56
```

fgets3.c

- 作用: 演示 `fgets()` 结合手动去除换行符的用法
- 功能: 从标准输入循环读取字符串, 手动检查并去除末尾的换行符, 直到遇到空行或 EOF, 每次读取后使用 `puts()` 输出
- 演示效果:

```
1  /* fgets3.c  -- using fgets() */
2  #include <stdio.h>
3  #define STLEN 10
4  int main(void)
5  {
6      char words[STLEN];
7      int i;
8
9      puts("Enter strings (empty line to quit):");
10     while (fgets(words, STLEN, stdin) != NULL
11            && words[0] != '\n')
12     {
13         i = 0;
14         while (words[i] != '\n' && words[i] != '\0')
15             i++;
16         if (words[i] == '\n')
17             words[i] = '\0';
18         else // must have words[i] == '\0'
19             while (getchar() != '\n')
20                 continue;
21         puts(words);
22     }
23     puts("done");
24     return 0;
```

```

1 gcc fgets3.c -o fgets3 && echo "Hello`nWorld`nC`nProgramming`n" | ./fgets3
2 Enter strings (empty line to quit):
3 Hello
4 World
5 C
6 Programming
7 done

```

fgets2.c

- 作用: 演示 `fgets()` 和 `fputs()` 在循环中读取多行输入的用法
- 功能: 从标准输入循环读取字符串, 直到遇到空行或 EOF, 每次读取后立即使用 `fputs()` 输出
- 演示效果:

```

1 /* fgets2.c -- using fgets() and fputs() */
2 #include <stdio.h>
3 #define STLEN 10
4 int main(void)
5 {
6     char words[STLEN];
7
8     puts("Enter strings (empty line to quit):");
9     while (fgets(words, STLEN, stdin) != NULL && words[0] != '\n')
10         fputs(words, stdout);
11     puts("Done.");
12
13     return 0;
14 }

```

```

1 gcc fgets2.c -o fgets2 && echo "Hello`nWorld`nC`nProgramming`n" | ./fgets2
2 Enter strings (empty line to quit):
3 Hello
4 World
5 C
6 Programming
7 Done.

```

name1.c

- 作用: 演示使用 `gets()` 从标准输入读取字符串
- 功能: 提示用户输入姓名并读取到字符数组中, 然后输出问候语 (注意: `gets()` 不安全, 可能导致缓冲区溢出, 建议使用 `fgets()`)
- 演示效果:

```
1  /* name1.c -- reads a name */
2  #include <stdio.h>
3  #define MAX 81
4  int main(void)
5  {
6      char name[MAX]; /* allot space */
7
8      printf("Hi, what's your name?\n");
9      gets(name);
10     // gets_s(name, MAX); /* place string into name array */
11     printf("Nice name, %s.\n", name);
12
13     return 0;
14 }
```

```
1  gcc name1.c -o name1 && echo "Alice" | ./name1
2  Hi, what's your name?
3  Nice name, Alice.
```

mod_str.c

- 作用: 演示字符串的修改与统计标点符号数量
- 功能: 从用户输入读取一行字符串, 去除末尾换行符后将所有字符转换为大写, 并统计其中的标点符号数量
- 演示效果:

```
1  /* mod_str.c -- modifies a string */
2  #include <stdio.h>
3  #include <string.h>
4  #include <ctype.h>
5  #define LIMIT 81
6  void ToUpper(char *);
```

```

7  int PunctCount(const char *);
8
9  int main(void)
10 {
11     char line[LIMIT];
12     char * find;
13
14     puts("Please enter a line:");
15     fgets(line, LIMIT, stdin);
16     find = strchr(line, '\n');    // look for newline
17     if (find)                    // if the address is not NULL,
18         *find = '\0';            // place a null character there
19     ToUpper(line);
20     puts(line);
21     printf("That line has %d punctuation characters.\n",
22           PunctCount(line));
23
24     return 0;
25 }
26
27 void ToUpper(char * str)
28 {
29     while (*str)
30     {
31         *str = toupper(*str);
32         str++;
33     }
34 }
35
36 int PunctCount(const char * str)
37 {
38     int ct = 0;
39     while (*str)
40     {
41         if (ispunct(*str))
42             ct++;
43         str++;
44     }
45
46     return ct;
47 }

```

```
1 gcc mod_str.c -o mod_str && echo "Hello, World!" | ./mod_str
2 Please enter a line:
3 HELLO, WORLD!
4 That line has 2 punctuation characters.
```

join_chk.c

- 作用: 演示在连接两个字符串前检查可用空间的用法
- 功能: 提示用户输入花名和虫名, 使用 `strcat()` 和 `strncat()` 将附加字符串连接到原字符串前, 先检查剩余空间, 避免缓冲区溢出
- 演示效果:

```
1  /* join_chk.c -- joins two strings, check size first */
2  #include <stdio.h>
3  #include <string.h>
4  #define SIZE 30
5  #define BUGSIZE 13
6  char * s_gets(char * st, int n);
7  int main(void)
8  {
9      char flower[SIZE];
10     char addon[] = "s smell like old shoes.";
11     char bug[BUGSIZE];
12     int available;
13
14     puts("What is your favorite flower?");
15     s_gets(flower, SIZE);
16     if ((strlen(addon) + strlen(flower) + 1) <= SIZE)
17         strcat(flower, addon);
18     puts(flower);
19     puts("What is your favorite bug?");
20     s_gets(bug, BUGSIZE);
21     available = BUGSIZE - strlen(bug) - 1;
22     strncat(bug, addon, available);
23     puts(bug);
24
25     return 0;
26 }
27 char * s_gets(char * st, int n)
28 {
29     char * ret_val;
```

```

30     int i = 0;
31
32     ret_val = fgets(st, n, stdin);
33     if (ret_val)
34     {
35         while (st[i] != '\n' && st[i] != '\0')
36             i++;
37         if (st[i] == '\n')
38             st[i] = '\0';
39         else // must have words[i] == '\0'
40             while (getchar() != '\n')
41                 continue;
42     }
43     return ret_val;
44 }

```

```

1 gcc join_chk.c -o join_chk && echo "Rose`nBeetle" | ./join_chk
2 What is your favorite flower?
3 Roses smell like old shoes.
4 What is your favorite bug?
5 Beetles smel

```

fgets1.c

- 作用: 演示 `fgets()` 和 `fputs()` 的基本用法
- 功能: 从标准输入读取字符串并分别使用 `puts()` 和 `fputs()` 输出, 展示它们的输出效果
- 演示效果:

```

1  /* fgets1.c -- using fgets() and fputs() */
2  #include <stdio.h>
3  #define STLEN 14
4  int main(void)
5  {
6      char words[STLEN];
7
8      puts("Enter a string, please.");
9      fgets(words, STLEN, stdin);
10     printf("Your string twice (puts(), then fputs()):\n");
11     puts(words);
12     fputs(words, stdout);
13     puts("Enter another string, please.");

```

```

14     fgets(words, STLEN, stdin);
15     printf("Your string twice (puts(), then fputs()):\n");
16     puts(words);
17     fputs(words, stdout);
18     puts("Done.");
19
20     return 0;
21 }

```

```

1 gcc fgets1.c -o fgets1 && echo "HelloWorld`nCProgramming" | ./fgets1
2 Enter a string, please.
3 Your string twice (puts(), then fputs()):
4 HelloWorld
5
6 HelloWorld
7 Enter another string, please.
8 Your string twice (puts(), then fputs()):
9 CProgramming
10
11 CProgramming
12 Done.

```

copy3.c

- 作用: 演示 `strncpy()` 函数的用法及其在限制复制长度时的效果
- 功能: 提示用户输入以字母 q 开头的单词, 使用 `strncpy()` 将单词复制到固定长度的数组中, 并手动添加字符串结束符以防止溢出
- 演示效果:

```

1  /* copy3.c -- strncpy() demo */
2  #include <stdio.h>
3  #include <string.h> /* declares strncpy() */
4  #define SIZE 40
5  #define TARGSIZE 7
6  #define LIM 5
7  char * s_gets(char * st, int n);
8
9  int main(void)
10 {
11     char qwords[LIM][TARGSIZE];
12     char temp[SIZE];

```

```

13     int i = 0;
14
15     printf("Enter %d words beginning with q:\n", LIM);
16     while (i < LIM && s_gets(temp, SIZE))
17     {
18         if (temp[0] != 'q')
19             printf("%s doesn't begin with q!\n", temp);
20         else
21         {
22             strncpy(qwords[i], temp, TARGSIZE - 1);
23             qwords[i][TARGSIZE - 1] = '\0';
24             i++;
25         }
26     }
27     puts("Here are the words accepted:");
28     for (i = 0; i < LIM; i++)
29         puts(qwords[i]);
30
31     return 0;
32 }
33
34 char * s_gets(char * st, int n)
35 {
36     char * ret_val;
37     int i = 0;
38
39     ret_val = fgets(st, n, stdin);
40     if (ret_val)
41     {
42         while (st[i] != '\n' && st[i] != '\0')
43             i++;
44         if (st[i] == '\n')
45             st[i] = '\0';
46         else // must have words[i] == '\0'
47             while (getchar() != '\n')
48                 continue;
49     }
50     return ret_val;
51 }

```

```
1 gcc copy3.c -o copy3 && echo "queen`nquick`nquiz`nquack`nquiet" | ./copy3
2 Enter 5 words beginning with q:
3 Here are the words accepted:
4 queen
5 quick
6 quiz
7 quack
8 quiet
```

copy2.c

- 作用: 演示 `strcpy()` 在字符串中部分复制的用法
- 功能: 将字符串常量复制到目标字符串的指定位置，展示复制后的整体效果与返回值的差异
- 演示效果:

```
1  /* copy2.c -- strcpy() demo */
2  #include <stdio.h>
3  #include <string.h>    // declares strcpy()
4  #define WORDS  "beast"
5  #define SIZE 40
6
7  int main(void)
8  {
9      const char * orig = WORDS;
10     char copy[SIZE] = "Be the best that you can be.";
11     char * ps;
12
13     puts(orig);
14     puts(copy);
15     ps = strcpy(copy + 7, orig);
16     puts(copy);
17     puts(ps);
18
19     return 0;
20 }
```

```
1 gcc copy2.c -o copy2 && ./copy2
2 beast
3 Be the best that you can be.
4 Be the beast
5 beast
```

copy1.c

- 作用: 演示 `strcpy()` 函数的基本用法
- 功能: 提示用户输入以字母 q 开头的单词，检查并复制符合条件的单词到二维数组中，最后输出接收的结果
- 演示效果:

```
1  /* copy1.c -- strcpy() demo */
2  #include <stdio.h>
3  #include <string.h> // declares strcpy()
4  #define SIZE 40
5  #define LIM 5
6  char * s_gets(char * st, int n);
7
8  int main(void)
9  {
10     char qwords[LIM][SIZE];
11     char temp[SIZE];
12     int i = 0;
13
14     printf("Enter %d words beginning with q:\n", LIM);
15     while (i < LIM && s_gets(temp, SIZE))
16     {
17         if (temp[0] != 'q')
18             printf("%s doesn't begin with q!\n", temp);
19         else
20         {
21             strcpy(qwords[i], temp);
22             i++;
23         }
24     }
25     puts("Here are the words accepted:");
26     for (i = 0; i < LIM; i++)
27         puts(qwords[i]);
28 }
```

```

29     return 0;
30 }
31
32 char * s_gets(char * st, int n)
33 {
34     char * ret_val;
35     int i = 0;
36
37     ret_val = fgets(st, n, stdin);
38     if (ret_val)
39     {
40         while (st[i] != '\n' && st[i] != '\0')
41             i++;
42         if (st[i] == '\n')
43             st[i] = '\0';
44         else // must have words[i] == '\0'
45             while (getchar() != '\n')
46                 continue;
47     }
48     return ret_val;
49 }

```

```

1 gcc copy1.c -o copy1 && echo "queen`nquick`nquiz`nquack`nquiet" | ./copy1
2 Enter 5 words beginning with q:
3 Here are the words accepted:
4 queen
5 quick
6 quiz
7 quack
8 quiet

```

compack.c

- 作用: 演示 `strcmp()` 函数的返回值含义
- 功能: 比较不同字符串并输出 `strcmp()` 的返回结果, 展示相等、前者小于后者、前者大于后者的情况
- 演示效果:

```

1 /* compack.c -- strcmp returns */
2 #include <stdio.h>
3 #include <string.h>

```

```

4  int main(void)
5  {
6
7      printf("strcmp(\"A\", \"A\") is ");
8      printf("%d\n", strcmp("A", "A"));
9
10     printf("strcmp(\"A\", \"B\") is ");
11     printf("%d\n", strcmp("A", "B"));
12
13     printf("strcmp(\"B\", \"A\") is ");
14     printf("%d\n", strcmp("B", "A"));
15
16     printf("strcmp(\"C\", \"A\") is ");
17     printf("%d\n", strcmp("C", "A"));
18
19     printf("strcmp(\"Z\", \"a\") is ");
20     printf("%d\n", strcmp("Z", "a"));
21
22     printf("strcmp(\"apples\", \"apple\") is ");
23     printf("%d\n", strcmp("apples", "apple"));
24
25     return 0;
26 }

```

```

1  gcc compback.c -o compback && ./compback
2  strcmp("A", "A") is 0
3  strcmp("A", "B") is -1
4  strcmp("B", "A") is 1
5  strcmp("C", "A") is 1
6  strcmp("Z", "a") is -1
7  strcmp("apples", "apple") is 1

```

compare.c

- 作用: 演示使用 `strcmp()` 比较字符串并进行循环判断
- 功能: 程序提示用户输入答案, 使用 `strcmp()` 与预设字符串进行比较, 直到输入正确为止
- 演示效果:

```

1  /* compare.c -- this will work */
2  #include <stdio.h>
3  #include <string.h> // declares strcmp()

```

```

4
5 #define ANSWER "Grant"
6 #define SIZE 40
7 char * s_gets(char * st, int n);
8
9 int main(void)
10 {
11     char try[SIZE];
12
13     puts("Who is buried in Grant's tomb?");
14     s_gets(try, SIZE);
15     while (strcmp(try, ANSWER) != 0)
16     {
17         puts("No, that's wrong. Try again.");
18         s_gets(try, SIZE);
19     }
20     puts("That's right!");
21
22     return 0;
23 }
24
25
26 char * s_gets(char * st, int n)
27 {
28     char * ret_val;
29     int i = 0;
30
31     ret_val = fgets(st, n, stdin);
32     if (ret_val)
33     {
34         while (st[i] != '\n' && st[i] != '\0')
35             i++;
36         if (st[i] == '\n')
37             st[i] = '\0';
38         else // must have words[i] == '\0'
39             while (getchar() != '\n')
40                 continue;
41     }
42     return ret_val;
43 }

```

```
1 gcc compare.c -o compare && echo "Grant" | ./compare
2 Who is buried in Grant's tomb?
3 That's right!
```

arrchar.c

- 作用: 演示指针数组与二维字符数组在存储字符串时的差异
- 功能: 定义一个指针数组存储字符串常量, 以及一个二维字符数组存储字符串副本, 比较它们的输出和内存占用
- 演示效果:

```
1 // arrchar.c -- array of pointers, array of strings
2 #include <stdio.h>
3 #define SLEN 40
4 #define LIM 5
5 int main(void)
6 {
7     const char *mytalents[LIM] = {
8         "Adding numbers swiftly",
9         "Multiplying accurately", "Stashing data",
10        "Following instructions to the letter",
11        "Understanding the C language"
12    };
13    char yourtalents[LIM][SLEN] = {
14        "Walking in a straight line",
15        "Sleeping", "Watching television",
16        "Mailing letters", "Reading email"
17    };
18    int i;
19
20    puts("Let's compare talents.");
21    printf ("%s %s\n", "My Talents", "Your Talents");
22    for (i = 0; i < LIM; i++)
23        printf("%s %s\n", mytalents[i], yourtalents[i]);
24    printf("\nsizeof mytalents: %zd, sizeof yourtalents: %zd\n",
25        sizeof(mytalents), sizeof(yourtalents));
26
27    return 0;
28 }
```

```

1 gcc arrchar.c -o arrchar && ./arrchar
2 Let's compare talents.
3 My Talents                                Your TalentsX
4 Adding numbers swiftly                    Walking in a straight line
5 Multiplying accurately                    SleepingX
6 Stashing dataX                            Watching television
7 Following instructions to the letter      Mailing letters
8 Understanding the C language              Reading email
9
10 sizeof mytalents: 40, sizeof yourtalents: 200

```

addresses.c

- 作用: 演示字符串在不同存储方式下的地址差异
- 功能: 定义字符串常量和字符数组，分别输出它们的内存地址，比较直接使用字符串字面量、字符数组、指针常量以及宏定义的地址
- 演示效果:

```

1 // addresses.c -- addresses of strings
2 #define MSG "I'm special."
3
4 #include <stdio.h>
5 int main()
6 {
7     char ar[] = MSG;
8     const char *pt = MSG;
9     printf("address of \"I'm special\": %p \n", "I'm special");
10    printf("          address ar: %p\n", ar);
11    printf("          address pt: %p\n", pt);
12    printf("          address of MSG: %p\n", MSG);
13    printf("address of \"I'm special\": %p \n", "I'm special");
14
15    return 0;
16 }

```

```
1 gcc addresses.c -o addresses && ./addresses
2 of "I'm special": 00007ff740bfa00d
3         address ar: 000000d4b8bff82b
4         address pt: 00007ff740bfa000
5         address of MSG: 00007ff740bfa000
6 address of "I'm special": 00007ff740bfa00d
```

WOW!Side A

Don't be a fool!

mesg = Don't be a fool!; &mesg = 0000007e3f5ff7e8; value = 00007ff707fca000

copy = Don't be a fool!; © = 0000007e3f5ff7e0; value = 00007ff707fca000